Simulation Interoperability Standards Organization

"Simulation Interoperability & Reuse through Standards"

SISO-REF-080-2023

Reference for High Level Architecture (HLA) Product Development Group (PDG) Library Nomenclature

Version 1.0

16 February 2023

SAC Approved: 01/04/2023 **EXCOM Approved: 02/16/2023**

Prepared by:
HLA PDG

Copyright © 2023 by the Simulation Interoperability Standards Organization, Inc.

P.O. Box 781238
Orlando, FL 32878-1238, USA
All rights reserved.

Reproduction and distribution of this document in whole or in part by any medium is permitted. Appropriate acknowledgement of the source and ownership of the material should be made with any such reproduction and distribution.

Please note that this document may be revised periodically. The latest edition will be made available at the SISO website at no charge. The document on the SISO website is considered to be the definitive version.

SISO Inc. Board of Directors
P.O. Box 781238
Orlando, FL 32878-1238, USA

Revision History

Version	Section	Date (MM/DD/YYY)	Description
Initial Version	All	02/16/2023	

Copyright © 2023 SISO. All rights reserved.

Table of Contents

1 INTRODUCTION 5
1.1 Purpose 5
1.2 Scope 5
1.3 Objectives 5
1.4 Intended Audience 5
1.5 Acknowledgements 5
2 REFERENCES 6
2.1 SISO Documents 6
2.2 Other Documents 7
3 DEFINITIONS 7
4 ACRONYMS AND ABBREVIATIONS 8
5 Library Names and Compiler Versions 8
5.1 Visual Studio Caveats 9
5.2 GNU Compiler Collection (GCC) Caveats – GCC 5, GCC 6, GCC 7 10
5.2.1 GCC 7 10

List of Tables

Table 5.1: Library Base Name 8
Table 5.2: Existing Compiler Version Markers 8

Copyright © 2023 SISO. All rights reserved.

1 INTRODUCTION

The High Level Architecture (HLA) provides a common architecture for distributed modeling and simulation. The HLA Federate Interface Specification (IEEE 1516.1-2023), the second in a family of three related HLA documents, defines the standard services of and interfaces to the HLA runtime infrastructure (RTI). In support of dynamic link compatibility between RTI C++ implementations, the HLA Federate Interface Specification specifies the use of a nomenclature format for RTI C++ library file names to be defined by this document. The nomenclature format allows for the specification of compilers, compiler versions, target operating systems, and any other characteristics intrinsic to dynamic link compatibility.

1.1 Purpose

The HLA Federate Interface Specification includes the initial set of RTI C++ library file names conformant with the nomenclature described in this document. The HLA standard changes less frequently than compiler versions and software library naming conventions. Keeping the nomenclature information up to date with the current state of compilers and operating systems is critical to continued dynamic link compatibility between RTI implementations. Maintaining this dynamic information within the IEEE standardization process is impractical. This SISO reference product maintains the most up to date RTI C++ library nomenclature information.

1.2 Scope

This reference product applies to C++ libraries for all versions of the HLA standard beginning with the 2023 version.

1.3 Objectives

This reference product is intended to prevent misidentification of RTI software libraries and their incompatibility with specific compilers. It also serves as a flexible way to adapt to changing characteristics of dynamic link compatibility which can change more frequently than is feasible for updating the HLA Interface Specification.

1.4 Intended Audience

This reference product is intended for HLA federate developers and federation engineers.

1.5 Acknowledgements

At the time this product was submitted to the Standards Activity Committee (SAC) for approval, the High Level Architecture (HLA) PDG had the following membership and was assigned the following SAC Technical Area Director.

PDG Leadership

- Randy Saunders (Chair)
- Björn Möller (Vice-Chair)
- Dr. Katherine L. Morse (Secretary)

SAC Technical Area Director: Simon Skinner

Document Editors

- Rules: Dr. Katherine L. Morse
- Federate Interface Specification: Doug Wood
- Object Model Template: Robert Lutz
- APIs: Mikael Karlsson
- XML: David L. Drake

Copyright © 2023 SISO. All rights reserved.

- Nomenclature: Dr. Katherine L. Morse

PDG Members

- Dr. Curtis Blais
- Jake Borah
- Dr. Uwe Dobrindt
- David L. Drake
- Aaron Dubois
- Matthew Figueroa
- Paul Gustavson
- Dr. Jean-Louis Igarza
- Kyle Isakson
- Mikael Karlsson
- Patrice Le Leydour
- Reed Little
- Robert Lutz
- Lance Marrou
- Mark McCall
- Björn Möller
- Dr. Katherine L. Morse
- David Ronnfeldt
- Jose Ruiz
- Randy Saunders
- Graham Shanks
- Keith Snively
- Tom van den Berg
- Rene Verhage
- Stefan Vrieler
- Douglas Wood
- Simone Youngblood

2 REFERENCES

2.1 SISO Documents

The following SISO documents were used in generating the policies and procedures defined herein. When the following documents are superseded by an approved revision and that causes a conflict with this document, the revision of the below-referenced documents shall supersede this document. These documents are available by through the SISO web site at https://www.sisostds.org/.

Copyright © 2023 SISO. All rights reserved.

Document Number	Title
N/A	

2.2 Other Documents

Document Number	Title
IEEE 1516.1-2023	IEEE Standard for Modeling and Simulation (M&S) High Level Architecture (HLA)—Federate Interface Specification
Microsoft	"C++ binary compatibility between Visual Studio 2015, 2017, and 2019," https://docs.microsoft.com/en-us/cpp/porting/binary-compat-2015-2017?view=vs-2019
Stack Overflow	"Is it safe to link C++17, C++14, and C++11 objects," https://docs.microsoft.com/en-us/cpp/porting/binary-compat-2015-2017?view=vs-2019
N/A	GNU, "Porting to GCC 7," https://www.gnu.org/software/gcc/gcc-7/porting_to.html

3 DEFINITIONS

Term	Definition
Authorizer	An interface used to determine if a specific set of service invocations (*Connect*, *Create Federation Execution*, *Destroy Federation Execution*, and *Join Federation Execution*) are allowed for the federate, given its credentials. [IEEE]
Federate	An application that may be or is currently coupled with other software applications under a federation object model (FOM) Document Data (FDD) and a runtime infrastructure (RTI). *See also*: **federate application**. [IEEE]
Federate application	An application that supports the High Level Architecture (HLA) interface to a runtime infrastructure (RTI) and that is capable of joining a federation execution. A federate application may join the same federation execution multiple times or may join multiple federation executions. However, each time a federate application joins a federation execution, it is creating a new joined federate. [IEEE]
Federation	A named set of federate applications and a common federation object model (FOM) that are used as a whole to achieve some specific objective. [IEEE]
Logical time	The representation of discrete points on the time axis. Logical time is used to represent a federate's time position and the timestamps of messages. [IEEE]
Runtime infrastructure	The software that provides common interface services during a High Level Architecture (HLA) federation execution for synchronization and data exchange. [IEEE]

Copyright © 2023 SISO. All rights reserved.

4 ACRONYMS AND ABBREVIATIONS

Acronym/Abbr	Definition
ABI	Application Binary Interface
FDD	Federation Object Model (FOM) Document Data
FOM	Federation Object Model
GCC	GNU Compiler Collection
HLA	High Level Architecture
IEEE	Institute for Electrical and Electronics Engineers
PDG	Product Development Group
SAC	Standards Activities Committee
SISO	Simulation Interoperability Standards Organization
RTI	Runtime Infrastructure
XML	Extensible Markup Language

5 Library Names and Compiler Versions

The following nomenclature is used for naming HLA libraries starting with IEEE 1516-2023.

lib<library base name>1516_2023<compiler version marker>[32-bit marker][debug marker].<platform extension> (where 32-bit marker is "_32")

Table 5.1 identifies the libraries provided with all RTIs and their associated base names.

Table 5.1: Library Base Name

Library	Base Name
RTI API implementation	rti
Logical time implementation	fedtime
Authorizer implementation	authorizer

If a 32-bit version of the library is provided, the library name will include the 32-bit marker, “32.” 64-bit library names will not include a marker.

Table 5.2 provides the compiler version markers in use at the time of publication of this document.

Table 5.2: Existing Compiler Version Markers

Compiler Name	Platform	Marker	Compatible With
Visual Studio 2022	Windows	vc143	Applications built with Visual Studio 2022 and later
Visual Studio 2019	Windows	vc142	Applications built with Visual Studio 2019 and later
Visual Studio 2017	Windows	vc141	Applications built with Visual Studio 2017 and later
Visual Studio 2015	Windows	vc140	Applications built with Visual Studio 2015 and later

Copyright © 2023 SISO. All rights reserved.

XCode 5	Mac OS	clang5	Applications built with XCode 5 and later
XCode 6	Mac OS	clang6	Applications built with XCode 6 and later
XCode 7	Mac OS	clang7	Applications built with XCode 7 and later
XCode 8	Mac OS	clang8	Applications built with XCode 8 and later
XCode 9	Mac OS	clang9	Applications built with XCode 9 and later
XCode 10	Mac OS	clang10	Applications built with XCode 10 and later
XCode 11	Mac OS	clang11	Applications built with XCode 11 and later
XCode 12	Mac OS	clang12	Applications built with XCode 12 and later
XCode 13	Mac OS	clang13	Applications built with XCode 13 and later
GCC 5		gcc5	Applications built with GCC 5 and later, up to, but not including GCC 7; see caveats in section 5.2.1
GCC 6		gcc6	Applications built with GCC 6 and later, up to, but not including GCC 7; see caveats in section 5.2
GCC 7		gcc7	Applications built with GCC 7 and later
GCC 8		gcc8	Applications built with GCC 8 and later
GCC 9		gcc9	Applications built with GCC 9 and later
GCC 10		gcc10	Applications built with GCC 10 and later
GCC 11		gcc11	Applications built with GCC 11 and later
GCC 12		gcc12	Applications built with GCC 12 and later

If a debug version of the library is provided, its name will include the debug marker, 'd.'

The platform extension will be “.so” for Linux libraries, “.dylib” for Mac OS, and “.dll” for Windows libraries. For example, the names for 32-bit libraries built for Windows with Visual Studio 2019 would be:

- librti1516_2023vc142_32.dll
- libfedtime1516_2023vc142_32.dll
- libauthorizer1516_2023vc142_32.dll

5.1 Visual Studio Caveats

It is possible to use libraries built with older versions of these toolsets (Visual Studio 2015, Visual Studio 2017, and Visual Studio 2019) with applications built with newer versions. However, libraries built with newer versions cannot be used with applications built with older versions. For example, a federate application built using the 2019 toolset can use an RTI library built using the 2017 toolset.

The *Visual Studio Redistributable* that your application uses has a similar binary-compatibility restriction. The *Visual Studio Redistributable* version must be at least as new as the toolset used to build the application.

Copyright © 2023 SISO. All rights reserved.

5.2 GNU Compiler Collection (GCC) Caveats – GCC 5, GCC 6, GCC 7

It is possible to use libraries built with older versions of these tool chains (GCC 5, GCC 6, and GCC 7) with applications built with newer versions. However, libraries built with newer versions cannot be used with applications built with older versions. For example, a federate application built using the GCC 6 tool chain can use an RTI library built using GCC 5 tool chain.

The application must use a libstdc++.so runtime library at least as recent as the one used when building the application.

5.2.1 GCC 7

GNU reports the following issue with respect to porting to GCC 7, "GCC 7 changes the name mangling for a conversion operator that returns a type using the abi_tag attribute, see PR 71712 (https://gcc.gnu.org/bugzilla/show_bug.cgi?id=71712). This affects code that has conversions to std::string, for example:

```
struct A {
  operator std::string() const;
};
```

Code using such conversions might fail to link if some objects are compiled with GCC 7 and some are compiled with older releases. [GNU]"

Copyright © 2023 SISO. All rights reserved.